Lab 7: Fuzzing for Input Validation Bugs

Akash Singh, PES2UG24CS042, Section A
Software Engineering, PES University
Repo: github.com/SETAPESU26/L6_L7_38_elevator_dispatch

The program under test is `elevator_dispatch.py` (the handout calls it `processor.py`). It has three functions: `parse_request` turns "5->9" into (5, 9), `total_travel` turns "1,5,3,9" into the total floors moved, and `load_percent` turns "450 of 630" into a percentage.

Tools: Python 3.14.3, pytest 9.2.0, Hypothesis 6.170.14 (installed with `pip install hypothesis pytest`).

Step 2: Run the original program

The sample inputs work. Typing 5-9 instead of 5->9 crashes the program with a ValueError.

```
PS D:\asd\SE\lab7> python elevator_dispatch.py
==== Running parse_request ====
Enter a request as FROM->TO (e.g. 5->9): 5->9
From and To: (5, 9)

==== Running total_travel ====
Enter floors visited as CSV (e.g. 1,5,3,9): 1,5,3,9
Floors Travelled: 12

==== Running load_percent ====
Enter load as 'X of Y' (e.g. 450 of 630): 450 of 630
Load Percent: 71
PS D:\asd\SE\lab7> python elevator_dispatch.py
==== Running parse_request ====
Enter a request as FROM->TO (e.g. 5->9): 5-9
Traceback (most recent call last):
  File "D:\asd\SE\lab7\elevator_dispatch.py", line 30, in <module>
    main()
    ~~~~^^
  File "D:\asd\SE\lab7\elevator_dispatch.py", line 16, in main
    result = parse_request(user_input)
  File "D:\asd\SE\lab7\elevator_dispatch.py", line 2, in parse_request
    start, end = text.split('->')
    ^^^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)
PS D:\asd\SE\lab7>
```

Step 3: Hypothesis tests

Each function has a no-crash test that runs 500 random inputs: any text, None, and short strings built from digits, '-', '>', ',', spaces and the letters in "of". Plain random text almost never contains -> or " of ", so without the last strategy the fuzzer would rarely get past split(). Each function also has a test that valid input still gives the right answer.

```
from hypothesis import given, settings, strategies as st
from elevator_dispatch import parse_request, total_travel, load_percent

# random text that looks a bit like real input, so the fuzzer gets past split()
messy = st.text(alphabet="0123456789-> ,of\t", max_size=20)
any_input = st.text() | st.none() | messy


@settings(max_examples=500)
@given(any_input)
def test_parse_request_no_crash(s):
    try:
        parse_request(s)
    except Exception as e:
        assert False, f"parse_request crashed with: {e!r}"


@settings(max_examples=500)
@given(any_input)
def test_total_travel_no_crash(s):
    try:
        total_travel(s)
    except Exception as e:
        assert False, f"total_travel crashed with: {e!r}"


@settings(max_examples=500)
@given(any_input)
def test_load_percent_no_crash(s):
    try:
        load_percent(s)
    except Exception as e:
        assert False, f"load_percent crashed with: {e!r}"


# valid input should still give the right answer

@given(st.integers(-5, 200), st.integers(-5, 200))
def test_parse_request_valid(a, b):
    assert parse_request(f"{a}->{b}") == (a, b)


@given(st.lists(st.integers(-5, 200), min_size=1, max_size=15))
def test_total_travel_valid(floors):
    expected = sum(abs(floors[i + 1] - floors[i]) for i in range(len(floors) - 1))
    assert total_travel(",".join(map(str, floors))) == expected


@given(st.integers(0, 2000), st.integers(1, 2000))
def test_load_percent_valid(load, cap):
    assert load_percent(f"{load} of {cap}") == load * 100 // cap
```

Step 4: Tests failing on the original code

All three no-crash tests fail. The valid-input tests pass.

```
PS D:\asd\SE\lab7> python -m pytest test_elevator_dispatch.py -v --tb=no
============================= test session starts =============================
platform win32 -- Python 3.14.3, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\akash singh\AppData\Local\Python\pythoncore-3.14-64\python.exe
hypothesis profile 'default'
rootdir: D:\asd\SE\lab7
plugins: anyio-4.15.1, hypothesis-6.170.14, platformdirs-4.13.5
collecting ... collected 6 items

test_elevator_dispatch.py::test_parse_request_no_crash FAILED            [ 16%]
test_elevator_dispatch.py::test_total_travel_no_crash FAILED             [ 33%]
test_elevator_dispatch.py::test_load_percent_no_crash FAILED             [ 50%]
test_elevator_dispatch.py::test_parse_request_valid PASSED               [ 66%]
test_elevator_dispatch.py::test_total_travel_valid PASSED                [ 83%]
test_elevator_dispatch.py::test_load_percent_valid PASSED                [100%]

=========================== short test summary info ===========================
FAILED test_elevator_dispatch.py::test_parse_request_no_crash - ExceptionGrou...
FAILED test_elevator_dispatch.py::test_total_travel_no_crash - ExceptionGroup...
FAILED test_elevator_dispatch.py::test_load_percent_no_crash - ExceptionGroup...
======================== 3 failed, 3 passed in 1.17s ========================
PS D:\asd\SE\lab7>
```

Part of the full output, showing the three smallest failing inputs Hypothesis found for parse_request: '->', '' and None.



```
PS D:\asd\SE\lab7> python -m pytest test_elevator_dispatch.py
_____________________ test_parse_request_no_crash ______________________
  + Exception Group Traceback (most recent call last):
  |   File "D:\asd\SE\lab7\test_elevator_dispatch.py", line 10, in test_parse_request_no_crash
  |     @given(any_input)
  |                    ^^
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "D:\asd\SE\lab7\test_elevator_dispatch.py", line 13, in test_parse_request_no_crash
    |     parse_request(s)
    |     ~~~~~~~~~~~~~~^^^
    |   File "D:\asd\SE\lab7\elevator_dispatch.py", line 3, in parse_request
    |     return int(start), int(end)
    |                ~~~^^^^^^^
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "D:\asd\SE\lab7\test_elevator_dispatch.py", line 15, in test_parse_request_no_crash
    |     assert False, f"parse_request crashed with: {e!r}"
    | AssertionError: parse_request crashed with: ValueError("invalid literal for int() with base 10: ''")
    | assert False
    | Failing test case: test_parse_request_no_crash(
    |     s='->',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "D:\asd\SE\lab7\test_elevator_dispatch.py", line 13, in test_parse_request_no_crash
    |     parse_request(s)
    |     ~~~~~~~~~~~~~~^^^
    |   File "D:\asd\SE\lab7\elevator_dispatch.py", line 2, in parse_request
    |     start, end = text.split('->')
    |     ^^^^^^^^^^
    | ValueError: not enough values to unpack (expected 2, got 1)
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "D:\asd\SE\lab7\test_elevator_dispatch.py", line 15, in test_parse_request_no_crash
    |     assert False, f"parse_request crashed with: {e!r}"
    | AssertionError: parse_request crashed with: ValueError('not enough values to unpack (expected 2, got 1)')
    | assert False
    | Failing test case: test_parse_request_no_crash(
    |     s='',  # or any other generated value
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "D:\asd\SE\lab7\test_elevator_dispatch.py", line 13, in test_parse_request_no_crash
    |     parse_request(s)
    |     ~~~~~~~~~~~~~~^^^
    |   File "D:\asd\SE\lab7\elevator_dispatch.py", line 2, in parse_request
    |     start, end = text.split('->')
    |                  ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "D:\asd\SE\lab7\test_elevator_dispatch.py", line 15, in test_parse_request_no_crash
    |     assert False, f"parse_request crashed with: {e!r}"
    | AssertionError: parse_request crashed with: AttributeError("'NoneType' object has no attribute 'split'")
    | assert False
    | Failing test case: test_parse_request_no_crash(
    |     s=None,  # or any other generated value
    | )
PS D:\asd\SE\lab7>
```

Bugs found

The first eight came from Hypothesis. In each function the first error hides the ones after it, so I tried a few more inputs by hand while fixing and found the last four.

Function	Input	Error in original code
parse_request	`None`	AttributeError (None has no split)
parse_request	`''`	ValueError: not enough values to unpack
parse_request	`'->'`	ValueError: invalid literal for int()
total_travel	`None`	AttributeError
total_travel	`''`	ValueError: invalid literal for int()
load_percent	`None`	AttributeError
load_percent	`''`	ValueError: not enough values to unpack
load_percent	`' of '`	ValueError: invalid literal for int()
load_percent	`'450 of 0'`	ZeroDivisionError (found by hand)
load_percent	`'-50 of 630'`	no crash, returns -8 (found by hand)
parse_request	`'1->2->3'`	ValueError: too many values (found by hand)
total_travel	`'1,,3'`	ValueError: invalid literal (found by hand)

Step 5: Fix

Each function now returns None for bad input instead of crashing. It checks that the input is a string, that split() gave exactly two parts (or that the floor list is not empty), and uses a helper to_int() that returns None when the text is not a whole number. load_percent also rejects a negative load and a capacity of 0 or less. main() prints "invalid input" when it gets None back.

```
def to_int(value):
    # returns None instead of raising when the text is not a whole number
    try:
        return int(value.strip())
    except ValueError:
        return None


def parse_request(text):
    if not isinstance(text, str):
        return None
    parts = text.split('->')
    if len(parts) != 2:
        return None
    start, end = to_int(parts[0]), to_int(parts[1])
    if start is None or end is None:
        return None
    return start, end


def total_travel(floors_csv):
    if not isinstance(floors_csv, str) or not floors_csv.strip():
        return None
    floors = [to_int(f) for f in floors_csv.split(',')]
    if None in floors:
        return None
    return sum(abs(floors[i + 1] - floors[i]) for i in range(len(floors) - 1))


def load_percent(text):
    if not isinstance(text, str):
        return None
    parts = text.split(' of ')
```

```
    if len(parts) != 2:
        return None
    load, capacity = to_int(parts[0]), to_int(parts[1])
    if load is None or capacity is None:
        return None
    if load < 0 or capacity <= 0:
        return None
    return load * 100 // capacity


def show(label, result):
    if result is None:
        print(label, "invalid input")
    else:
        print(label, result)


def main():
    print("==== Running parse_request ====")
    user_input = input("Enter a request as FROM->TO (e.g. 5->9): ")
    show("From and To:", parse_request(user_input))

    print("\n==== Running total_travel ====")
    user_input = input("Enter floors visited as CSV (e.g. 1,5,3,9): ")
    show("Floors Travelled:", total_travel(user_input))

    print("\n==== Running load_percent ====")
    user_input = input("Enter load as 'X of Y' (e.g. 450 of 630): ")
    show("Load Percent:", load_percent(user_input))

if __name__ == "__main__":
    main()
```

Step 5: Run the fixed program

Valid input gives the same answers as before. The bad inputs now print "invalid input".

```
PS D:\asd\SE\lab7> python elevator_dispatch.py
==== Running parse_request ====
Enter a request as FROM->TO (e.g. 5->9): 5->9
From and To: (5, 9)

==== Running total_travel ====
Enter floors visited as CSV (e.g. 1,5,3,9): 1,5,3,9
Floors Travelled: 12

==== Running load_percent ====
Enter load as 'X of Y' (e.g. 450 of 630): 450 of 630
Load Percent: 71
PS D:\asd\SE\lab7> python elevator_dispatch.py
==== Running parse_request ====
Enter a request as FROM->TO (e.g. 5->9): 5-9
From and To: invalid input

==== Running total_travel ====
Enter floors visited as CSV (e.g. 1,5,3,9): 1,,3
Floors Travelled: invalid input

==== Running load_percent ====
Enter load as 'X of Y' (e.g. 450 of 630): 450 of 0
Load Percent: invalid input
PS D:\asd\SE\lab7>
```

Step 6: Tests on the fixed code

All 6 tests pass.

```
PS D:\asd\SE\lab7> python -m pytest test_elevator_dispatch.py -v
============================= test session starts =============================
platform win32 -- Python 3.14.3, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\akash singh\AppData\Local\Python\pythoncore-3.14-64\python.exe
hypothesis profile 'default'
rootdir: D:\asd\SE\lab7
plugins: anyio-4.15.1, hypothesis-6.170.14, platformdirs-4.13.5
collecting ... collected 6 items

test_elevator_dispatch.py::test_parse_request_no_crash PASSED            [ 16%]
test_elevator_dispatch.py::test_total_travel_no_crash PASSED             [ 33%]
test_elevator_dispatch.py::test_load_percent_no_crash PASSED             [ 50%]
test_elevator_dispatch.py::test_parse_request_valid PASSED               [ 66%]
test_elevator_dispatch.py::test_total_travel_valid PASSED                [ 83%]
test_elevator_dispatch.py::test_load_percent_valid PASSED                [100%]

============================== 6 passed in 2.98s ==============================
PS D:\asd\SE\lab7>
```